Random Forest＋LLMによるプロンプトインジェクション検出手法の再考：分類器トリアージと選択的LLM呼び出しによる追試評価

北島 琉斗[1,2,a)]　吉塚 創也[1,b)]　宮本 耕平[2,c)]　高橋 健志[2,d)]　櫻井 幸一[1,e)]

概要：近年，Random Forest などの機械学習分類器と大規模言語モデル（LLM）を組み合わせたプロンプトインジェクション攻撃の検出手法が提案されている。本研究では，先行研究で提案した Random Forest ＋ LLM 統合手法を再検討し，大規模データセットを用いて追試評価を行った。その結果，分類器単独の検出性能はデータ分布の変化に大きく左右され，実運用環境における信頼性に課題があることを確認した。そこで，分類器で低リスク入力を事前に選別し，残る高リスク入力のみを LLM で評価するトリアージ方式を検討した。PromptShield データセットを用いた評価では，再現率および F1 スコアをほぼ維持しつつ，LLM 呼び出し回数を 23.2%削減できた。さらに，適応的攻撃を想定した分析を通じて，本方式の有効性と限界を考察する。

キーワード：プロンプトインジェクション，LLM-as-a-Judge，トリアージ，分布変化，適応的攻撃

Revisiting Prompt Injection Detection with Random Forests and Large Language Models: A Follow-up Evaluation of Classifier-Based Triage with Selective LLM Invocation

Ryuto Kitajima[1,2,a)]　Souya Yoshizuka[1,b)]　Kohei Miyamoto[2,c)]　Takeshi Takahashi[2,d)]
Kouichi Sakurai[1,e)]

Abstract: Recent studies have explored prompt injection detection methods that combine machine learning classifiers, such as random forests, with large language models (LLMs). In this study, we revisit our previously proposed Random Forest-LLM hybrid method and conduct a follow-up evaluation using a large-scale dataset. The results show that the standalone classifier is highly sensitive to distribution shifts, raising concerns about its reliability in real-world deployments. We therefore examine a triage approach in which the classifier first filters low-risk inputs and the LLM evaluates only the remaining high-risk inputs. Experiments on the PromptShield dataset show that this approach reduces the number of LLM calls by 23.2% while largely preserving recall and the F1 score. We also analyze the approach under adaptive attacks and discuss its effectiveness and limitations.

Keywords: prompt injection, LLM-as-a-Judge, triage, distribution shift, adaptive attack

[1] 九州大学
Kyushu University
[2] 国立研究開発法人情報通信研究機構
National Institute of Information and Communications Technology
[a)] kitajima.ryuto.146@s.kyushu-u.ac.jp
[b)] syosiduka@gmail.com
[c)] k.miyamoto831@nict.go.jp
[d)] takeshi_takahashi@nict.go.jp
[e)] sakurai@inf.kyushu-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

大規模言語モデル（Large Language Model: LLM）を組み込んだアプリケーションは，利用者の入力と開発者の指示を同じ自然言語の文脈で処理する．攻撃者はこの仕組みを悪用して，システム指示の上書き，非公開プロンプトの取得，本来とは異なる処理の実行を要求できる．この攻撃はプロンプトインジェクション（Prompt Injection: PI）と呼ばれ，攻撃と防御の定式化やベンチマークの整備が進んでいる [1].

入力時の PI 検出には，軽量な機械学習分類器と LLM による判定（LLM-as-a-Judge）が用いられる．分類器は安価で高速だが，語彙やデータセット固有の手掛かりを学習しやすく，入力分布が変わると性能が落ちる [2]．LLM judge は入力の文脈や意図を踏まえて判定できるが，全入力を送れば費用と遅延が入力数に比例して増える．判定の一貫性にも問題が残る [3], [4]．近年のサーベイは，入力前処理，モデル強化，システム構成を組み合わせた防御を整理し，配備時には安全性と遅延・費用のトレードオフを考慮する必要があると指摘している [5].

吉塚ら [6] は，OpenAI の埋め込み表現と Random Forest からなる分類器を，LLM による自己評価と組み合わせた複数段階の PI 検出方式を提案した．同研究で構築したデータセットでは，分類器単独の場合より検出性能が向上した．入力分布が変化した場合の性能と，LLM 呼び出し回数・検出性能の関係は評価されていない．本研究では，この統合構成を大規模データセット上で追試し，分類器を低リスク入力の選別に用いる．低リスクと判定した入力だけをバイパスし，それ以外は固定した LLM judge へ送る．評価では，LLM judge を省略できる入力の範囲と，省略によって増える見逃しを別々に測定する．

本研究の目的は，分類器と LLM judge を組み合わせる運用条件を，見逃しと呼び出し回数の両面から明らかにすることである．貢献は 3 点ある．第 1 に，偽バイパス率に上限を設け，その範囲内で LLM 呼び出しを省くトリアージ規則を定めた．第 2 に，PromptShield[7] の公式 test から評価集合と重ならない較正集合を作成した．この分割を用いて分類器の validation–test 間の性能差を調べ，LLM 呼び出し回数が 23.2%減ることを確認した．第 3 に，外部ソースとホワイトボックス型パディング攻撃を別々に評価し，ストリーム監視で検出できる分布変化と，検出できない攻撃を確認した．入力単位ゲート，監査，セッション隔離についても捕捉率と追加呼び出しを報告する．図 1 に処理手順を示す．本方式では，分類器の権限を低リスク入力に対するバイパス許可に限定し，バイパス条件を満たさない入力は LLM judge へ送る．実装と再現用資料は匿名リポジトリ[*1]で公開している．

2. 関連研究と本稿の評価項目

2.1 分類器による PI 検出

埋め込み表現を Random Forest や XGBoost で分類する PI 検出器では，高い分類性能が報告されている [8]．同じデータセットをランダム分割すると，テンプレート，語彙，収集過程の特徴が学習側とテスト側に残りやすい．PromptShield は，実運用を想定し，低い誤検知率で検出性能を評価する PI 検出ベンチマークである [7]．NotInject は，PI に関連する語を含む良性入力を使って過剰防御を評価する [9]．本稿では Accuracy と F1 に加え，データソースを変えたときの性能，悪性入力のバイパス率，LLM 呼び出し率を測る．

2.2 先行する Random Forest + LLM 統合

先行研究 [6] は，OpenAI 埋め込みを入力とする Random Forest を第 1 段階，LLM の自己評価を第 2 段階，根拠の構造化表示を第 3 段階に置いた．同研究で構築したデータセットでは，分類器単独より検出精度が約 60%向上したと報告している．同研究の評価対象は構築データセット内の検出性能であり，未知ソースでの性能，分類器と LLM の入力ごとの役割分担，全件を LLM で判定する方式に対する呼び出し削減数は評価項目に含まれていない．

本追試では，先行研究の統合構成を出発点として，実運用で分類器に許可できる処理の範囲を検討する．評価対象を大規模データと外部ソースへ広げ，分類器の導入によって増える見逃しと LLM 呼び出し回数の関係を測る．分類器に由来する偽バイパス率を制約しながら，LLM 呼び出しを削減できる条件を評価する．

本追試における各要素の役割を表 1 に示す．タイトルの「Random Forest + LLM」は追試対象の先行構成を指す．LLM 呼び出し回数を 23.2%削減した実験では，単語・文字 TF–IDF と Logistic Regression（LR）をルータに用いた．LR は CPU 上で決定的に動作し，ホワイトボックス型攻撃の分析に必要な語の重みを取得できる．本稿では Random Forest と LR の優劣を比較しない．評価対象は，スコアを出力する分類器に共通して適用できるトリアージ規則である．

2.3 LLM ルーティングと judge の信頼性

FrugalGPT は，質問ごとに LLM を選び分けるモデルカスケードによって費用と性能を調整する [10]．本稿で分類器が許可する処理は，検証済みソースの低スコア入力を良性としてバイパスすることに限られる．スコアが閾値を超えた入力は固定 judge へ送るため，分類器の判定境界が影

[*1] https://anonymous.4open.science/r/cost-awarePI-triage-58E5

© 2026 Information Processing Society of Japan

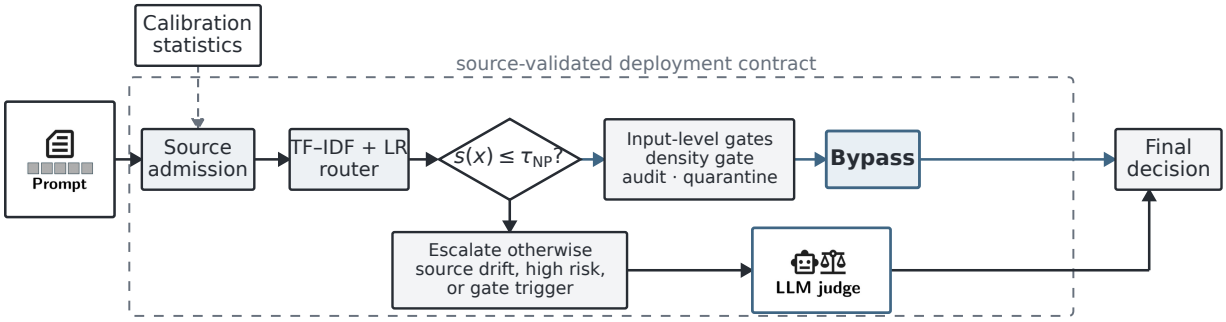


図 1 選択的 LLM 呼び出しの処理手順．検証済みソースの低リスク入力だけをバイパスし，残りは LLM judge へ送る．

表 1 先行研究と本追試の役割の違い

項目	先行研究	本追試
軽量モデル	埋め込み＋ Random Forest	TF–IDF ＋ LR
分類器の用途	攻撃・良性の判定	judge を省略する入力の選別
評価指標	検出精度	偽バイパス率，呼び出し率
分布変化	評価なし	ソース監視と外部評価
適応的攻撃	評価なし	ホワイトボックス型パディング攻撃

響する範囲はバイパス候補に限られる．LLM-as-a-Judge には位置バイアスや判定の不一致がある [3], [11]．性能評価では人手で付与されたラベルを真値とし，judge の出力との一致を測る．

3. 選択的 LLM 呼び出し手法

3.1 対象と脅威モデル

本稿では，利用者プロンプトが，システム指示の上書き，タスクの乗っ取り，プロンプト漏えい，秘密情報の取得を直接試みているかを二値分類する．ツール実行の成否，RAG に混入した間接 PI，多言語の長文，画像・音声入力は扱わない．バイパス対象の入力ソースについて，防御者がラベル付き履歴またはレッドチームデータを取得できる状況を想定する．

攻撃者は TF–IDF ルータと閾値を知っており，良性らしい語を加えてルータスコアを下げられる．較正ラベル，ソース監視の参照窓，LLM judge は操作できないと仮定する．全入力を固定 judge で判定する方式を基準とし，分類器の導入によって生じる悪性バイパス率を，検証済みソース内で所定の上限以下に抑える．全 PI の検出は保証しない．

3.2 ルーティング規則

入力を x，悪性ラベルを $Y = 1$，ルータスコアを $s(x) \in [0,1]$ とする．時刻 t に入力ソースを受理しているかを A_t，適応攻撃ゲートの出力を $g(x)$，監査または隔離による送信指示を $q(x)$ で表す．図 1 の 4 条件をすべて満たす入力に限って judge を省略する．この判定を $B_t(x)$ で表す．

$$B_t(x) = \mathbf{1}\{A_t = 1,\ s(x) \le \tau_{\mathrm{NP}}, g(x) = q(x) = 0\}. \quad (1)$$

$$\pi(x) = \begin{cases} \text{bypass}, & B_t(x) = 1, \\ \text{judge}, & \text{otherwise}. \end{cases} \quad (2)$$

ゲート，監査，隔離はいずれもバイパス候補を judge へ送るための機構であり，追加によってバイパス領域が広がることはない．呼び出し削減率は，ソース受理と NP 閾値からなる基本規則で測定する．適応攻撃向けの対策は 6 章で個別に評価する．

3.3 偽バイパス率を制約する閾値

較正データ中の悪性入力が低スコア領域へ入る確率を ϵ 以下に制約し，この制約の下でバイパス率を最大化する．

$$\begin{aligned} \tau^* &= \arg\max_{\tau} \Pr[s(X) \le \tau], \\ \text{s.t.} \quad & \Pr[s(X) \le \tau \mid Y = 1] \le \epsilon. \end{aligned} \quad (3)$$

悪性較正例 n_+ 件のスコアを $z_1, \ldots, z_{n_+}$ とし，スコアが候補閾値以下となる悪性例の件数を $k_\tau = \#\{i : z_i \le \tau\}$ とする．二項比率の片側 Clopper–Pearson 上限 [12] を U_{CP} とすると，

$$U_{\mathrm{CP}}(k_\tau, n_+; 1-\alpha) \le \epsilon \quad (4)$$

を満たす最大閾値を選ぶ．許容件数は (n_+, ϵ, α) から定まり，対応する順序統計量の直下に閾値を置く．配備時の悪性入力と較正時の悪性入力を交換可能とみなせる場合，この手順は信頼度 $1-\alpha$ で悪性入力のバイパス確率を ϵ 以下に抑える片側非母数許容限界を与える．入力ソースが変

© 2026 Information Processing Society of Japan

わった場合や，攻撃者がスコアを操作する場合にはこの保証を適用できない．

本実験では $\epsilon = 0.005$，$\alpha = 0.05$ とした．PromptShield から LLM 評価用レコードと重ならない 10,000 件を取り出し，較正に用いた．悪性 2,560 件中 6 件までを許容すると，$\tau_{\mathrm{NP}} = 0.0208$ となる．同じ条件でバイパスを 1 件以上許可するには，悪性較正例が少なくとも 598 件必要である．較正偽バイパスを 1 件許す場合は 947 件，6 件では 2,366 件を要する．悪性較正例が 598 件未満のソースでは，この設定でバイパスを許可する閾値を得られない．

3.4 入力ソースの検査と LLM judge

較正時と異なる入力ソースに同じ閾値を使わないため，ルータスコア分布の Population Stability Index（PSI），プロンプト長の 2 標本 Kolmogorov–Smirnov 検定，PI 関連語の出現率差を監視する．PSI$\geq$ 0.30，KS 検定 $p < 0.01$，キーワード率差の絶対値 ≥ 0.15 のいずれかを満たしたソースは拒否し，全入力を judge へ送る．この 3 つの閾値は，外部ソースの正解ラベルを参照せず，試験的に設定した．

judge には元のプロンプトだけを渡し，ルータスコアは渡さない．gpt-4o-mini を temperature 0 で実行し，悪性判定と理由を JSON で取得した．judge の出力を JSON として解析できない場合はバイパスさせず，悪性判定とするか，人手確認に回す．本実験では解析失敗はなかった．

4. 評価方法

4.1 研究課題

本評価では 4 つの研究課題を設定した．RQ1 では，同一ソース内で偽バイパスを抑えつつ，LLM 呼び出しを削減できるかを調べる．RQ2 では，validation から test への移行と外部ソースへの適用によって，分類器の性能がどの程度低下するかを測る．RQ3 では，正解ラベルを用いないストリーム監視によって，較正時と異なるソースを拒否できるかを確認する．RQ4 では，低混入率の適応的攻撃が監視をすり抜ける条件と，入力単位の対策による捕捉数を調べる．

4.2 データセットと分割

使用したデータを表 2 に示す．PromptShield の train でルータを学習し，validation でハイパーパラメータと診断用閾値を決めた．公式 test から，自然比率 3,000 件，均衡 2,000 件，NP 較正 10,000 件を固定シードで重複なく抽出した．閾値の選択に評価用レコードは用いていない．外部評価には deepset[13] を用いた．良性の難例（hard negative）には NotInject[9] を，攻撃入力のみのストレス試験には Lakera Gandalf[14] を用いた．

PromptShield の公式分割間にも分布差がある．代表的な PI 関連語の出現率は，train/validation では約 35%だったのに対し，test では 14.5%だった．この分布差を評価に残すため，ランダムな再分割は行わなかった．自然比率 3,000 件には悪性入力が 823 件含まれる．

表 2 使用したデータセット

データセット	件数	悪性件数
PromptShield train	18,909	9,452
PromptShield validation	1,000	503
PromptShield test	23,516	6,486
deepset	662	263
NotInject	339	0
Gandalf	1,000	1,000

4.3 比較方法と指標

全入力を固定 judge へ送る LLM-only を基準とし，固定閾値または NP 較正閾値によって低リスク入力をバイパスする方式と比較する．主指標には，悪性入力に対するルータ由来の偽バイパス率と，システム全体の再現率（end-to-end 再現率）を用いる．Accuracy，Precision，Specificity，F1 も併記する．運用効率は，LLM 呼び出し率，入出力トークン数，実測遅延で評価する．固定 judge を用いたときの見逃しは

$$\mathrm{FN}_{\mathrm{total}} = \mathrm{FN}_{\mathrm{bypass}} + \mathrm{FN}_{\mathrm{judge|escalated}} \tag{5}$$

と分解する．ルータ由来の偽バイパスが 0 件であっても，judge による見逃しは第 2 項として残る．F1 については，3,000 件の評価標本を再標本化し，bootstrap 法による 95% 区間を算出した．偽バイパス率には Wilson 区間と片側 Clopper–Pearson 上限を併記した．

5. 評価結果

5.1 分類器単独性能の分布変化

TF–IDF + LR の F1 は validation で 0.940 だった．validation で定めた診断用閾値を test の自然比率 3,000 件に適用すると，F1 は 0.427，再現率は 0.375 まで低下した．gpt-4o-mini judge は Accuracy 0.885，再現率 0.683，Specificity 0.962，F1 0.766 だった．この結果は，validation で選んだ分類器の性能が，異なる入力分布で大幅に低下したことを示す．分類器を最終判定に用いるには，ソースごとの再評価が必要である．gpt-4o-mini judge の再現率も 0.683 にとどまり，LLM-only でも悪性入力の見逃しは残った．

5.2 同一ソース内の呼び出し削減

自然比率 3,000 件の結果を表 3 に示す．$\tau = 0.01$ では LLM 呼び出しが 16.9%減り，偽バイパスは 0 件だった．NP 較正閾値では呼び出しが 23.2%減り，悪性 823 件のうち 1 件（0.1%）が judge に送られずにバイパスされた．再現率は 0.683 から 0.682，F1 は 0.766 から 0.769 となった．F1 の bootstrap 95%区間は LLM-only で $[0.741, 0.790]$，NP 較正で $[0.745, 0.793]$ であり，大きく重なる．F1 差 0.003 を

© 2026 Information Processing Society of Japan

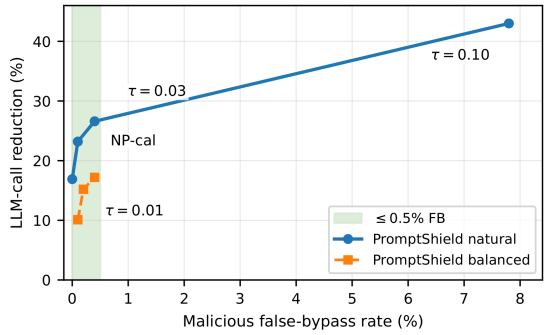


図 2 PromptShield 同一ソース内の悪性偽バイパス率と LLM 呼び出し削減率．緑色は偽バイパス率 0.5%以下の範囲である．

性能改善とは解釈しない．23.2%の呼び出し削減に伴う F1 の低下も，この標本では確認されなかった．

$\tau = 0.10$ では呼び出しを 43.0%減らせるものの，図 2 のとおり偽バイパス率は 7.8%まで増え，再現率も下がる．本稿の基本設定には，式 4 の偽バイパス率上限を満たす NP 較正点を用いた．この設定では，悪性入力のバイパス確率に上限を設けたうえで呼び出しを削減する．

同じ NP 閾値を均衡 2,000 件に適用した場合，削減率は 15.2%だった．呼び出しを省ける割合は，入力中の良性例の割合によって変わる．judge を gpt-4.1-mini へ替えると，LLM-only の F1 は 0.816，トリアージの F1 は 0.818 だった．呼び出し削減で生じた費用の余裕を，より高性能な judge の利用に充てる運用も考えられる．judge 間の性能差は，ルータの効果と分けて解釈する必要がある．

5.3 外部ソースへの適用

PromptShield で較正した閾値を外部データへそのまま適用すると，deepset の悪性例の 62.4%，Gandalf の 13.8%がバイパスされた．いずれも較正時に設定した 0.5%の上限を大きく超えており，この閾値は転用できない．表 4 の分布監視は両ソースを拒否した．全入力を judge へ送ったため，ルータ由来の偽バイパスは 0 件になった．良性例のみからなる NotInject も拒否された．NP 閾値だけを適用した場合には 73.5%の呼び出しを省けたが，監視後は全件が judge へ送られた．分布監視は保守的に動作し，分布差を検知したソースでは良性例の割合にかかわらず呼び出しを削減しない．

5.4 費用と遅延

NP 較正点では，LLM 呼び出しが 23.2%，judge 経路の総トークン数が 19.4%減った．バイパスされたプロンプトが平均より短いため，トークン数の減少率は呼び出し数より小さい．1 入力当たりの実測遅延は，gpt-4o-mini で 119 ms から 92 ms，gpt-4.1-mini で 148 ms から 114 ms へ短縮した．gpt-4o-mini の推定費用は 1,000 入力当たり 0.136 米ドルから 0.100 米ドルへ下がった．モデル価格は今後変わり得るため，本稿では呼び出し率とトークン数も，価格に依存しない費用指標として報告する．

6. 適応的攻撃に対する評価

6.1 低混入率のホワイトボックス型パディング攻撃

攻撃者がルータを知っていれば，負の重みを持つ良性語を Gandalf の攻撃文の前に加え，$s(x)$ を下げられる．具体的な回避文字列は公開しない．固定シードで変種を生成し，オフラインで評価した．PromptShield 1,980 件に攻撃 20 件を加え，攻撃混入率 1%のストリームを構成した．20 件すべてのスコアが τ_{NP} を下回った．PSI は 0.011 にとどまり，検出閾値を 0.10 まで下げてもストリーム監視は作動しなかった．

20 件すべてがルータのバイパス条件を満たしたうえ，混入率が 1%だったため，分布監視も変化を捉えなかった．バイパス候補を 5%の確率で監査しても，20 件のうち監査される攻撃は平均 1 件にとどまる．監査対象外の入力は judge へ届かない．同じ変種 60 件を judge へ直接送った場合の見逃しは 7 件（11.7%）であり，見逃しの大半は judge に到達する前に生じた．

6.2 入力単位ゲートと隔離

攻撃混入率 1%での対策を表 5 に示す．density gate は，良性判定に寄与する単語の重みがプロンプト前半に偏っている入力を judge へ送る．閾値は，同一ソースから抽出した非注入入力のみからなる holdout で較正した．固定パディング，共通語を乱数化した変種，攻撃ごとに異なる語を乱数化した変種のすべてで，20 件中 19 件を捕捉した．このゲートによって追加送信された良性入力は 0.5%だった．

固定 prefix gate の捕捉数は，固定パディングでは 19/20 件，乱数化した変種では 1/20 件だった．tail keyword gate は 17/20 件を捕捉し，良性入力の追加送信率は 13.8%だった．5%ランダム監査は，表に示したシードでは 2/20 件を捕捉した．100 シード平均の end-to-end 見逃し率は 95.1%であり，監査だけでは攻撃の大半を見逃した．同一 ID から攻撃が繰り返される場合には，監査で一度検出した後，その ID からの入力をすべて judge へ送るセッション隔離を適用した．20 回試行時の見逃し率は 96.8%から 73.6%へ下がった．攻撃者が ID を替える場合には，この効果を期待できない．

7. 考察

7.1 分類器に与える権限

validation では F1 0.940 であった分類器が，test では 0.427 まで低下した．同一分布の validation で得た F1 だけを根拠に，分類器を最終判定へ用いることはできない．

© 2026 Information Processing Society of Japan

表 3 PromptShield 自然比率 3,000 件における選択的 LLM 呼び出し．FB は悪性 823 件中のルータ由来偽バイパスである．

judge	ポリシー	呼出削減	Accuracy	Recall	Specificity	F1	FB
gpt-4o-mini	LLM-only	0.0%	0.885	0.683	0.962	0.766	0
gpt-4o-mini	$\tau = 0.01$	16.9%	0.888	0.683	0.965	0.769	0 (0.0%)
gpt-4o-mini	NP 較正	**23.2%**	0.888	0.682	0.966	0.769	1 (0.1%)
gpt-4o-mini	$\tau = 0.03$	26.6%	0.888	0.682	0.966	0.770	3 (0.4%)
gpt-4o-mini	$\tau = 0.10$	43.0%	0.881	0.650	0.968	0.750	64 (7.8%)
gpt-4.1-mini	LLM-only	0.0%	0.912	0.713	0.987	0.816	0
gpt-4.1-mini	NP 較正	**23.2%**	0.914	0.713	0.989	0.818	1 (0.1%)

表 4 ソース分布監視の結果．拒否時は全件 LLM judge へフォールバックする．

ソース	ポリシー／トリガ	呼出削減	Recall/Specificity	F1	ルータ FB
PromptShield holdout	NP ＋監視（受理）	23.2%	0.675	0.760	1 (0.2%)
deepset	NP のみ	80.8%	0.262	0.416	164 (62.4%)
deepset	NP ＋監視（PSI, KS）	0.0%	0.605	0.754	0
NotInject	NP のみ	73.5%	0.991	–	–
NotInject	NP ＋監視（PSI, KS）	0.0%	0.982	–	–
Gandalf	NP のみ	13.8%	0.803	0.891	138 (13.8%)
Gandalf	NP ＋監視（KS, keyword）	0.0%	0.897	0.946	0

本稿では，分類器の権限を低スコア入力のバイパス許可に限定した．この構成では，分類器の導入によって新たに生じた見逃しを偽バイパスとして直接数えられるため，LLM-only との差を明確に評価できる．同じ評価方法は，スコアを出力する他の分類器にも適用できる．

7.2 23.2%削減の適用条件

23.2%という削減率は，入力が PromptShield の較正データと同じ分布に従う場合に得られた値である．本方式を適用できるのは，入力を較正データと交換可能とみなせる検証済みソースに限られる．新しいソースでは，較正に必要な数の悪性例を確保できるまで，全入力を judge へ送るべきである．3 種類の監視指標は deepset と Gandalf を拒否したが，混入率 1%のホワイトボックス型攻撃には反応しなかった．ソース監視は，較正閾値を適用するかどうかをソース単位で決める．個々の入力には density gate，ランダム監査，セッション隔離，rate limit を併用し，judge の後段にも guardrail を設ける必要がある．後段防御の候補には，PromptArmor[15] や instruction hierarchy[16] がある．

7.3 LLM judge による性能上限

トリアージは LLM judge 自体の判定性能を改善しない．gpt-4o-mini の再現率は 0.683 であり，ルータによる偽バイパスを 0 件に抑えても，judge が見逃す入力は残る．gpt-4.1-mini に変更すると F1 は高くなったが，複数の judge を組み合わせる方法は評価していない．

7.4 妥当性への脅威

内部妥当性．本評価では既存データセットのラベルを真値とし，再注釈は行っていない．density gate の適応的攻撃評価も各変種 20 件に限られる．既存ラベルの誤りや標本数の少なさが評価値に影響する可能性がある．また，temperature を 0 に設定しても，API モデルの更新後に同じ judge 出力が得られるとは限らない．

外部妥当性．23.2%の削減率は PromptShield で得た値であり，評価対象も主として英語の直接 PI に限られる．間接 PI，ツール利用，RAG，長文，多言語，画像・音声を用いた攻撃は対象外である．未知のパディング，複数箇所に分散したパディング，ルータ再学習後の攻撃，良性文書の大量挿入には，今回の捕捉率を一般化できない．

8. おわりに

本研究では，先行研究の Random Forest ＋ LLM 型 PI 検出を大規模データで追試し，分類器の役割を再検討した．評価に用いた TF–IDF ＋ LR 分類器の F1 は，validation の 0.940 から test の 0.427 へ低下した．この結果を受け，分類器の権限を検証済みソースにおける低スコア入力のバイパス許可に限定した．PromptShield の自然比率 3,000 件では，NP 較正により LLM 呼び出し回数を 23.2%削減した．偽バイパスは悪性 823 件中 1 件であり，再現率は 0.683 から 0.682，F1 は 0.766 から 0.769 となった．

PromptShield の閾値を外部ソースへ転用すると偽バイパスが増え，混入率 1%のホワイトボックス型パディングはストリーム監視をすり抜けた．したがって，23.2%の削減は検証済みソースに限った結果であり，異なる分布や適

© 2026 Information Processing Society of Japan

表 5 1%適応攻撃混入ストリームにおける対策比較（$N = 2{,}000$）.

対策	捕捉数	適応攻撃 E2E 見逃し	呼出削減	良性追加送信
基本 NP トリアージ	0/20	100%	23.9%	0.0%
固定 prefix gate	19/20	5.0%	22.9%	0.0%
良性重み density gate	**19/20**	**5.0%**	22.9%	0.5%
tail keyword gate	17/20	15.0%	22.7%	13.8%
5%ランダム監査	2/20	90.0%	22.9%	–

応的攻撃に対する保証ではない．配備時にはソースごとに閾値を較正し，分布が変われば全入力を judge へ戻す必要がある．入力単位ゲート，ランダム監査，セッション隔離，後段防御も併用する．

今後は，複数ソースを用いて一般化性能を検証し，Random Forest，SLM，専用 guard model を同一の偽バイパス制約の下で比較する．評価対象も，間接 PI，RAG，多言語，長文，マルチモーダル入力へ広げる．複数 judge と後段 guardrail を組み合わせた構成についても評価する．

本稿の評価では，検証済みソースにおいて分類器を低リスク入力の選別に用いることで，偽バイパス率を制約しながら LLM judge の呼び出し回数を削減できた．

本研究は，Prompt Injection 検出において分類器を最終判定器として用いるのではなく，LLM judge を補助するトリアージ機構として利用する有効性を示した．

参考文献

[1] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847. USENIX Association, 2024. https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei.

[2] Max Fomin. When benchmarks lie: Evaluating malicious prompt classifiers under true distribution shift, 2026. https://arxiv.org/abs/2602.14161.

[3] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on LLM-as-a-Judge, 2024. https://arxiv.org/abs/2411.15594.

[4] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Yuan Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. In *International Conference on Learning Representations (ICLR)*, 2025. https://openreview.net/forum?id=G0dksFayVq.

[5] Tongcheng Geng, Zhiyuan Xu, Yubin Qu, and W. Eric Wong. Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies. *Computers, Materials & Continua*, 87(1):4, 2026. https://doi.org/10.32604/cmc.2025.074081.

[6] 吉塚 創也 and 櫻井 幸一. Random Forest による分類器と LLM を統合したプロンプトインジェクション攻撃検出手法の提案. **情報処理学会研究報告コンピュータセキュリティ（*CSEC*）**, 2026-CSEC-112(16):1–8, 2026. https://ipsj.ixsq.nii.ac.jp/records/2008537.

[7] Dennis Jacob, Hend Alzahrani, Zhanhao Hu, Basel Alomair, and David Wagner. PromptShield: Deployable detection for prompt injection attacks, 2025. https://arxiv.org/abs/2501.15145.

[8] Md. Ahsan Ayub and Subhabrata Majumdar. Embedding-based classifiers can detect prompt injection attacks. In *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024)*, pages 257–268, 2024. https://ceur-ws.org/Vol-3920/paper15.pdf.

[9] Hao Li and Xiaogeng Liu. InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models, 2024. https://arxiv.org/abs/2410.22770.

[10] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. https://openreview.net/forum?id=cSimKw5p6R.

[11] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023. https://openreview.net/forum?id=uccHPGDlao.

[12] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[13] deepset. deepset/prompt-injections dataset, 2024. https://huggingface.co/datasets/deepset/prompt-injections.

[14] Lakera AI. Lakera/gandalf_ignore_instructions dataset, 2024. https://huggingface.co/datasets/Lakera/gandalf_ignore_instructions.

[15] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. PromptArmor: Simple yet effective prompt injection defenses, 2025. https://arxiv.org/abs/2507.15219.

[16] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions, 2024. https://arxiv.org/abs/2404.13208.

© 2026 Information Processing Society of Japan

研究倫理

本研究では公開データセットのみを使用し，隔離した環境で評価した．実在サービスに対する攻撃は行っていない．ホワイトボックス型パディングは防御性能を調べるためにオフラインで生成し，攻撃にそのまま転用できる語列は本文および成果物に掲載しない．公開データセット中のプロンプトを API judge へ送信する際にも，秘密情報や個人情報を追加で収集していない．成果公開時には，CSEC 研究会のサイバーセキュリティ研究倫理チェックリストに基づき，データセットのライセンス，公開する攻撃コードの粒度，API 利用条件を再確認する．

© 2026 Information Processing Society of Japan